



ONE SEMESTER PROJECT

SYSTEM DESIGN SPECIFICS

24/06590

KALOKI CARRICK MWELII

BDS 3106

BACHELOR OF SCIENCE IN DATA SCIENCE

SUPERVISOR - LISPER KENDI

PROJECT TITLE : SMART BANKLYTICS

An Intelligent Data – Driven Bank Management and Customer Analytics System

1. INTRODUCTION

1.1 Purpose

This Software Design Specification (SDS) document provides a detailed technical design for the Smart Banklytics system. It describes the system architecture, internal modules, database structure, user interface layout, and technologies to be used during system development. This document serves as a blueprint for developers to implement the system efficiently and accurately according to the requirements specified in the Software Requirements Specification (SRS).

1.2 Scope

Smart Banklytics is a web-based intelligent banking system designed to automate banking operations, provide real-time analytics, and enhance customer experience. The system supports secure transactions, account management, fraud detection, and analytics visualization. This SDS focuses on the technical structure, system components, and implementation strategies of the Smart Banklytics platform.

2. SYSTEM OVERVIEW

Smart Banklytics follows a modular and layered system design approach. The system enables customers to manage their banking activities while administrators monitor financial performance and system security through analytical dashboards. It integrates transaction processing, data storage, analytics computation, and visualization into a unified web-based platform.

The system processes user requests through a web interface, performs business logic operations on the backend, stores and retrieves data from a relational database, and displays results through interactive dashboards and reports.

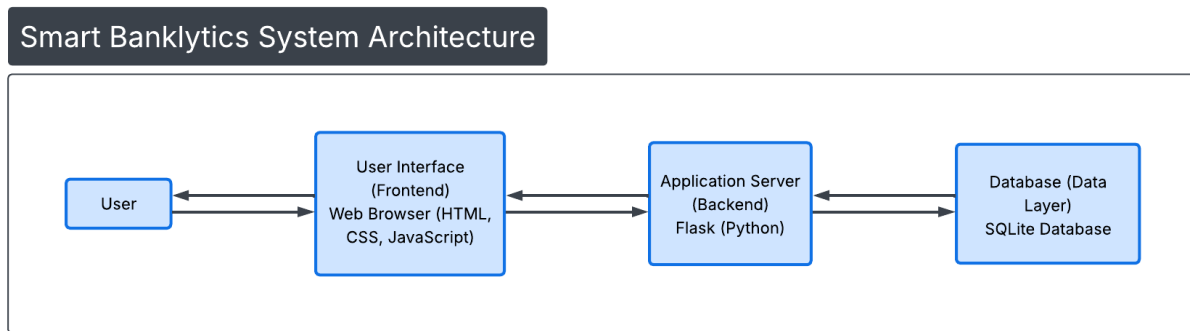
3. SYSTEM ARCHITECTURE DESIGN

3.1 Architecture Style

Smart Banklytics adopts a **Three-Tier Architecture**, which separates the system into three main layers:

1. **Presentation Layer (Frontend)**
2. **Application Layer (Backend)**
3. **Data Layer (Database)**

This architecture enhances scalability, maintainability, and security.



3.2 Architecture Description

Presentation Layer:

This layer provides the user interface and interacts directly with users. It is developed using HTML, CSS, and JavaScript. It includes pages such as login, dashboards, transaction pages, analytics displays, and report generation interfaces.

Application Layer:

This layer handles business logic and system processing. It is implemented using Python with the Flask framework. It manages authentication, transaction processing, analytics computation, fraud detection logic, and system validations.

Data Layer:

This layer manages persistent data storage using an SQLite relational database. It stores user records, transaction logs, analytics results, and system audit logs.

4. SYSTEM MODULE DESIGN

The system is divided into several functional modules, each responsible for a specific task.

4.1 User Authentication Module

Purpose:

Handles user login, authentication, authorization, and session management.

Inputs:

Username, password

Processing:

- Validates user credentials
- Assigns role-based access
- Manages active user sessions

Outputs:

- Secure login access
 - Role-based dashboard redirection
-

4.2 Customer Account Management Module

Purpose:

Allows customers to manage personal banking activities.

Inputs:

Deposit amount, withdrawal amount, transfer details

Processing:

- Balance validation
- Transaction recording
- Account updates

Outputs:

- Updated balances
 - Transaction confirmation
 - Account summaries
-

4.3 Administrator Management Module

Purpose:

Enables administrators to manage users and monitor system operations.

Inputs:

User creation data, account control commands

Processing:

- User creation and updates
- Account activation or suspension
- System monitoring

Outputs:

- Updated user accounts
 - System activity logs
-

4.4 Transaction Management Module

Purpose:

Processes and records all financial transactions securely.

Inputs:

Transaction requests

Processing:

- Transaction validation
- Balance verification
- Transaction execution
- Logging

Outputs:

- Transaction success/failure messages
 - Updated transaction logs
-

4.5 Analytics and KPI Monitoring Module

Purpose:

Generates analytical insights and performance metrics.

Inputs:

Transaction and account data

Processing:

- Data aggregation
- KPI calculation
- Chart generation

Outputs:

- Graphical dashboards
 - Statistical summaries
-

4.6 Fraud Detection Module

Purpose:

Detects suspicious transactions and prevents fraudulent activities.

Inputs:

Transaction patterns and frequency

Processing:

- Pattern analysis
- Anomaly detection
- Rule-based fraud detection

Outputs:

- Fraud alerts
 - Automatic account suspension
 - Admin notifications
-

4.7 Password Recovery Module

Purpose:

Allows users to reset forgotten passwords securely.

Inputs:

User verification details

Processing:

- Identity validation
- Secure password update

Outputs:

- Password reset confirmation
-

5. DATABASE DESIGN

5.1 Database Overview

The system uses an SQLite relational database to store all operational data. The database ensures data consistency, integrity, and security.

5.2 Entity Relationship Overview

The main entities include:

- Users
- Transactions

Each user may have multiple transactions, forming a **one-to-many relationship**.

5.3 Database Tables

5.3.1 Users Table

Stores user account and profile information.

Field	Description
id	Primary key
username	User login name
password	Encrypted password
first_name	User first name
last_name	User last name
id_number	National ID
account_number	Bank account number
account_type	Savings / Current / Student
balance	Account balance
role	User / Admin
status	Active / Suspended

5.3.2 Transactions Table

Stores all transaction details.

Field	Description
id	Primary key
account_number	Related user account
transaction_type	Deposit / Withdrawal / Transfer
amount	Transaction amount
date	Timestamp
from_account	Source account
to_account	Destination account

6. USER INTERFACE DESIGN

The system user interface is designed to be simple, intuitive, and responsive.

Main Pages Include:

- Login Page
- Customer Dashboard
- Admin Dashboard
- Transaction Management Page
- Analytics Dashboard
- Reports Page

Each interface is designed with usability and accessibility in mind to enhance user experience.

7. TECHNOLOGY STACK

Component Technology

Frontend HTML, CSS, JavaScript

Backend Python (Flask Framework)

Database SQLite

Data Analysis Pandas, NumPy

Visualization Matplotlib, Chart.js

Platform Web-based system

8. SYSTEM FLOW DESIGN

1. User logs into the system.
2. System authenticates credentials.
3. User accesses dashboard based on role.

4. Transactions are processed and validated.
5. Data is stored in the database.
6. Analytics engine processes stored data.
7. Dashboards and reports are generated.

9. SECURITY DESIGN CONSIDERATIONS

- Password encryption and hashing
- Role-based access control
- Secure session management
- Input validation
- Transaction verification

10. CONCLUSION

This Software Design Specification document defines the architectural structure, system modules, and database framework for Smart Banklytics. The design ensures modularity, security, scalability, and maintainability, enabling efficient implementation and future expansion of the system.